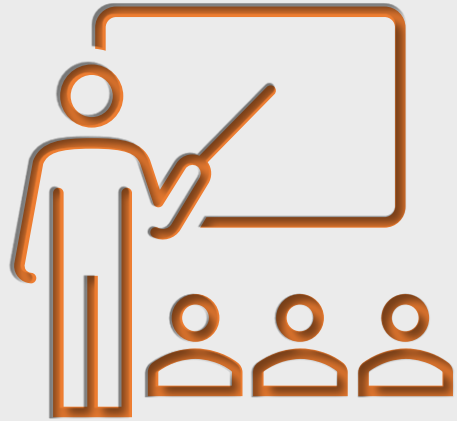




RÉVISIONS

Fondamentaux de la programmation
impérative & procédurale

LES BOUCLES BORNÉES

LES BOUCLES BORNÉES

- Rappels :

- Une boucle bornée est une boucle dont on connaît à l'avance le nombre de répétitions.
- Une boucle non bornée est implémentée par l'instruction **while**
- La boucle bornée peut également être implémentée par une instruction **while** mais également par l'instruction « spécialisée » **for i in range**

LES BOUCLES BORNÉES

- **Exemple d'implémentation en Python :**

- Le code suivant réalise une boucle de comptage de 10 à 100 par pas d'incrément de 2 :

```
1  # Valeurs des arguments de la fonction range() :
2  valDep = 10
3  valFin = 100
4  pas = 2
5
6  # Boucle de comptage :
7  for i in range(valDep, valFin, pas):
8      print(i, end = "-")
9
```

LES BOUCLES BORNÉES

- **Exercice 1 :**

- Compléter le programme ci-dessous qui a le même comportement que le programme précédent mais en utilisant l'instruction **while** au lieu de l'instruction **for i in range**

```
1 # Variables associées à la boucle :
2 valDep = 10
3 valFin = 100
4 pas = 2
5
6 # Boucle de comptage :
7
8 while(i < valFin):
9     print(i, end = "-")
10
11
```

LISTES & PARCOURS

LISTES & PARCOURS

- Rappels fondamentaux sur les listes :

- Les listes sont des variables structurées qui permettent de mémoriser plusieurs éléments de type différents (entiers, flottants, booléens, etc.).
- Les listes sont une structure de données itérable et ordonnée :
 - **Itérable**, car on peut parcourir tous les éléments d'une liste avec l'instruction **for elt in liste**
 - **Ordonnée**, car chaque élément est classé par un indice qui le rend directement accessible.

LISTES & PARCOURS

- Exemples d'instructions fondamentales sur les listes :

```
1 # Déclaration d'une liste :
2 listeVide = []
3 listeDemo = ["truc", "bidule", True, 5, -7.08]
4
5 # Accès à un élément par indice :
6 print("3 est à l'indice : " + str(listeDemo[3]))
7
8 # Longueur (nombre d'éléments) d'une liste :
9 print("Longueur de la liste : " + str(len(listeDemo)))
10
11 # Parours de tous les éléments d'une liste :
12 print("La liste contient : ", end = "")
13 for chose in listeDemo:
14     print(chose, end = "; ")
15
16 # Ajout d'un élément :
17 listeVide.append(9)
18 listeVide = listeVide + [3, 6, 2]
19 print("\nLa liste devient : ", listeVide)
```

```
3 est à l'indice : 5
Longueur de la liste : 5
La liste contient : truc; bidule; True; 5; -7.08;
La liste devient : [9, 3, 6, 2]
```


LISTES & PARCOURS

- **Programmation procédurale :**

- Il peut être maintenant intéressant de « consigner » notre code dans une fonction/procédure.
- Pour rappel, une procédure est une partie de code que l'on peut appeler à volonté.
- Durant l'appel de la fonction/procédure on peut éventuellement fournir un ou plusieurs paramètres afin que l'exécution soit adaptée aux données à traiter.

LISTES & PARCOURS

- Exemple d'une procédure de parcours d'une liste :

```
1 # Définition de la procédure :
2 def parcoursListe(listeParamètre):
3     for element in listeParamètre:
4         print(element, end = "-")
5
6 # Génération de la liste argument :
7 listeArgument = list(range(10, 50, 2))
8
9 # Appel de la procédure avec l'argument listeArgument :
10 parcoursListe(listeArgument)
```

```
>>> %Run '06-Procédure sur une liste.py'
10-12-14-16-18-20-22-24-26-28-30-32-34-36-38-40-42-44-46-48-
>>>
```

LISTES & PARCOURS

- Dans l'exemple précédent nous avons bien à faire une procédure, car cette dernière agit directement dans la console (l'affichage des valeurs contenues dans la liste).
- Par contre elle ne fournit/retourne pas de valeurs au programme.
- Une fonction se distingue d'une procédure par le fait qu'elle produit une ou plusieurs valeurs qui sont utilisables pour la suite du programme.

LISTES & PARCOURS

- Dans le prochain exemple et exercice, nous aurons une fonction qui retournera une valeur.
- Cette valeur étant la valeur maximale contenue dans une liste qui est donnée en argument durant l'appel de la fonction.

LISTES & PARCOURS

- Exercice 2 :

- Compléter le corps de la définition de la fonction.

```
1 # Recherche d'une valeur maximale :
2 # -----
3
4 # Définition de la fonction :
5 def maxiliste(liste):
6     """Fonction retournant la valeur maxi contenue
7     dans la liste."""
8
9     
10
11
12
13     return valMax
14
15 # Test de la fonction :
16 from random import shuffle
17 listeTest = (list(range(17)))
18 shuffle(listeTest) # Mélange aléatoire de la liste.
19 print(str(maxiliste(listeTest)) + " plus grande valeur de la liste : " + str(listeTest))
```

```
>>> %Run '08-Valeur maxi d'""'une liste.py'
16 plus grande valeur de la liste : [16, 6, 8, 7, 12, 2, 5, 15, 0, 10, 4, 3, 14, 13, 1, 9, 11]
>>>
```

LISTES & PARCOURS

- **Exercice 3 :**

- Compléter la fonction suivante qui retourne le nombre d'occurrences d'une valeur contenue dans une liste.

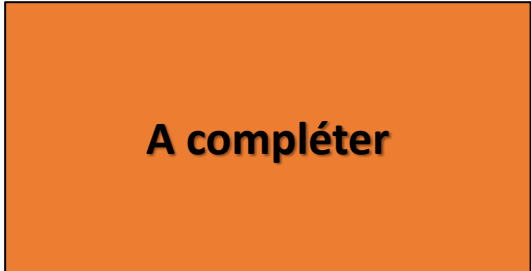
```
1 # Nombre d'occurrences d'une valeur dans une liste :
2 # -----
3 def nbOccurrences(val, liste):
4     """Fonction retournant le nombre de répétitions de la valeur
5     val contenue dans la liste donnée en 2e paramètre."""
6
7
8
9
10
11     return cpt
12
13 # Exemple :
14 from random import randint
15 listeDemo = []
16 for i in range(30):
17     listeDemo.append(randint(0, 10))
18 valOccurrence = 5
19 print("Il y a " + str(nbOccurrences(valOccurrence, listeDemo)) + " occurrence(s) de la valeur " + str(valOccurrence) + " dans la liste :\n\t " + str(listeDemo))
20
```

```
>>> %Run '09-Nbr occurrences valeur dans liste.py'
Il y a 5 occurrence(s) de la valeur 5 dans la liste :
[3, 0, 5, 6, 1, 0, 9, 4, 1, 0, 6, 1, 0, 6, 3, 6, 7, 9, 4, 5, 6, 9, 5, 2, 1, 4, 1, 5, 1, 5]
```

LISTES & PARCOURS

- Exercice 4 :

- Réaliser la définition de la fonction suivante :

```
1 def nbMaxListe(liste):
2     """Fonction retournant la valeur maxi et son nombre d'occurrences."""
3
4     
5
6
7
8
9
10
11     return valMax, cptMax
12
13 # Exemple :
14 from random import randint
15 listeDemo = []
16 for i in range(30):
17     listeDemo.append(randint(0, 10))
18 valMax, cptMax = nbMaxListe(listeDemo)
19 print("Il y a " + str(cptMax) + " occurrence(s) de la valeur maximale " + str(valMax) + " dans la liste :\n\t" + str(listeDemo))
```

```
>>> %Run '10-Nbr occurrences de la valeur max d''''une liste.py'
```

```
Il y a 4 occurrence(s) de la valeur maximale 10 dans la liste :
    [10, 8, 6, 6, 6, 5, 5, 10, 4, 0, 9, 2, 1, 7, 4, 3, 7, 9, 5, 2, 0, 8, 8, 6, 2, 1, 2, 0, 8, 10]
```

LISTES & PARCOURS

- Exercice 5 :

- Compléter la fonction :

```
1 def sommeListe(liste):  
2     """Fonction retournant la somme des valeurs d'une liste."""  
3  
4       
5  
6  
7  
8 # Exemple :  
9 from random import shuffle  
10 listeDemo = list(range(10))  
11 shuffle(listeDemo)  
12 somme = sommeListe(listeDemo)  
13 print("Somme des valeurs de la liste :", listeDemo, "est :", somme)
```

```
>>> %Run '11-Somme des valeurs d'""""une liste.py'
```

```
Somme des valeurs de la liste : [1, 9, 3, 8, 4, 5, 2, 0, 6, 7] est : 45
```


LISTES & PARCOURS

- Exercice 6 :

- Compléter la définition de la fonction du calcul de la moyenne :

```
1 def moyListe(liste):  
2     """Fonction qui retourne la moyenne des  
3         valeurs de la liste donnée en argument."""  
4  
5  
6  
7  
8  
9  
10  
11 # Exemple :  
12 from random import shuffle  
13 listeDemo = list(range(10))  
14 shuffle(listeDemo)  
15 moy = moyListe(listeDemo)  
16 print("La moyenne de la liste :", listeDemo, ":", moy)  
17
```

A compléter

```
>>> %Run '12-Moyenne des valeurs d''''une liste.py'  
La moyenne de la liste : [5, 0, 8, 4, 7, 2, 9, 1, 6, 3] : 4.5
```

FIN